UNIVERSIDAD PERUANA
CAYETANO HEREDIA



FACULTAD DE CIENCIAS E INGENIERÍA

ENTREGABLE N° 10

CURSO:

FUNDAMENTOS DE BIODISEÑO

ESTUDIANTES:

- OBISPO DIAZ, SHARON ACUARELA
- OLORTEGUI SANCHEZ, MARIA FERNANDA
- PALACIOS DOMINGUEZ, YACELLY BELEN
- ROSAS ROMAN, GERSON ADRIEL
- SANCHEZ ROJAS, ANAMI VALENTINA
- SANDOVAL BACA, RODRIGO EDSON

HORARIO:

2:00 - 6:00 PM

2026-2

ENTREGABLE N°10 - GRUPO 09

1. **Mecánica y fabricación**

- Para la fabricación de las piezas se utilizaron modelos digitales CAD, trabajando con Autodesk Inventor y Bambu Studio para la preparación y configuración de los archivos de impresión. Se utilizó una impresora Bambu Lab A1 Mini, empleando PLA como material de fabricación. Hasta el momento, la única pieza impresa ha sido la caja destinada a alojar el módulo ESP32, ya que los demás componentes se encuentran ubicados internamente en la estructura.

Registro fotográfico y comparación con el modelo CAD:

- Se realizó un registro fotográfico de la caja fabricada para comparar visualmente su geometría con el modelo CAD utilizado como referencia y comprobar la correspondencia general entre ambos.



Figura 1. Primera versión impresa de la caja para el módulo ESP32.

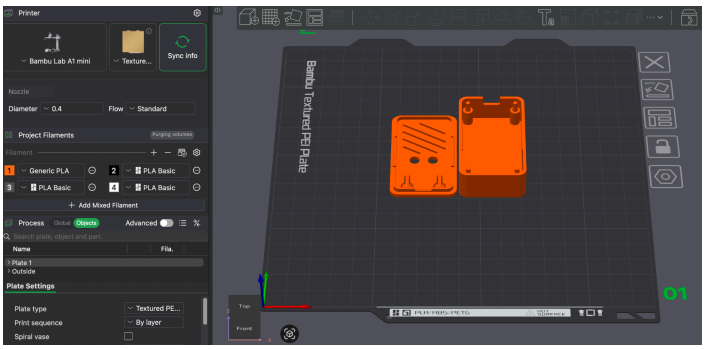


Figura 2. Proceso de impresión de la segunda versión de la caja para el módulo ESP32.

Inconvenientes durante la fabricación:

- Durante el proceso se presentó una confusión en la selección del ESP32, ya que inicialmente se consideró un modelo de 8 pines por lado, cuando el componente definitivo corresponde a uno de 15 pines por lado. Por este motivo, la primera caja impresa no se ajusta correctamente al módulo que se utilizará. Ante ello, se realizó la modificación correspondiente y la nueva pieza ya fue enviada a impresión. Esta reemplazará a la anterior tan pronto como esté disponible. Asimismo, si durante el ensamblaje se identifica la necesidad de incorporar otras piezas o realizar ajustes, se considerarán nuevas impresiones para garantizar un correcto montaje de los componentes.

2. Electrónica y sensores

Para la implementación del subsistema electrónico del corsé, se realizó el montaje del circuito base en un protoboard, integrando el microcontrolador ESP32 como unidad de procesamiento. Se establecieron las conexiones para los cuatro sensores sEMG. Para lograrlo, se diseñó un arreglo con dos baterías de 9V que proporcionan los voltajes +Vs y -Vs para amplificar las señales eléctricas generadas por la contracción de los músculos.

Una parte clave del armado fue proteger las entradas analógicas del ESP32. Como los módulos sEMG amplifican las señales usando una fuente de 9V, tuvimos que armar divisores de tensión en la salida de señal (SIG) de cada

sensor, en total 4, usando resistencias de 4.7kΩ y 10kΩ. Con esto logramos bajar el voltaje a un máximo de 3.3V, lo que nos permitió medir las contracciones musculares de forma segura y sin peligro de quemar los pines ADC del microcontrolador.

Las fotos muestran cómo quedaron distribuidos los componentes y el cableado inicial del sistema. Luego nos aseguramos que todo funcione correctamente usando el multímetro para medir los nodos de los divisores de tensión, con la finalidad de confirmar que el voltaje bajó antes de empezar a calibrar los umbrales de activación muscular en el monitor serial.

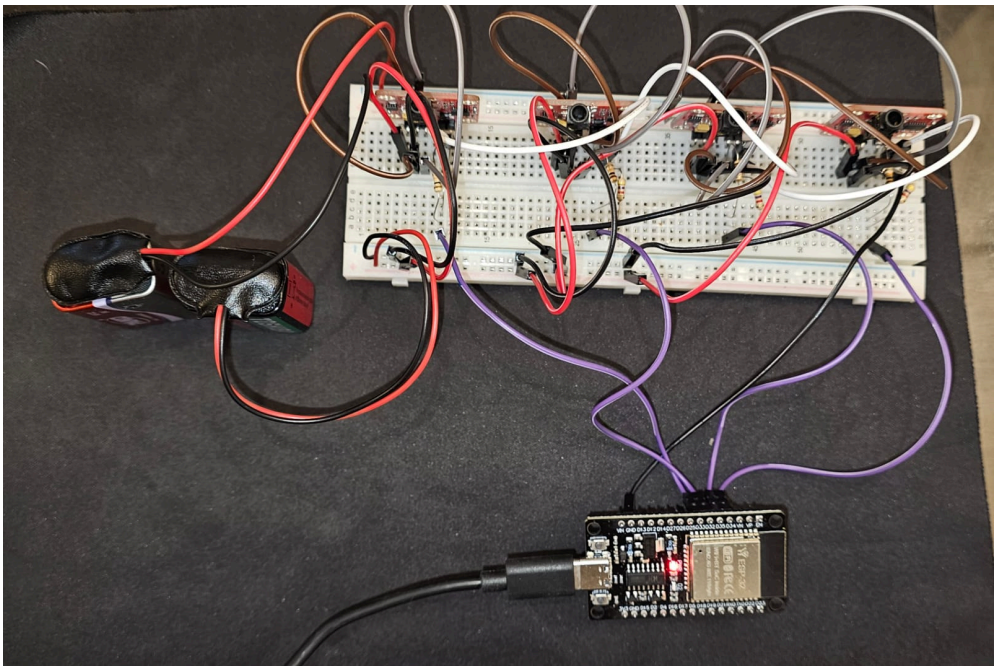


Figura 3. Vista superior del circuito base

3. Programación y control

3.1 Implementación del módulo de adquisición de señales sEMG:

- Se desarrolló un programa en Arduino IDE para la tarjeta DOIT ESP32 DEVKIT V1 que busca adquirir y procesar las señales provenientes de cuatro sensores de electromiografía superficial (sEMG).
- El programa realiza la lectura secuencial de cuatro canales analógicos, utilizando el convertidor analógico-digital del ESP32 con una resolución de 12 bits (valores comprendidos entre 0 y 4095). Integra un filtro

digital de media móvil exponencial (EMA) para atenuar el ruido de alta frecuencia y obtener la envolvente muscular suavizada, formateando los datos en tiempo real para el Serial Plotter de Arduino IDE.

Finalmente, los valores procesados son enviados mediante comunicación serial para la visualización en el Serial Plotter de Arduino IDE

3.2 Descripción de las funciones principales:

El código implementado comprende las siguientes etapas:

- **Configuración de los sensores:** Se asignaron los pines GPIO 34, 35, 32 y 33 del ESP32 para recibir las señalas analógicas de los sensores sEMG.
- **Inicialización del sistema:** Mediante la función *setup()*, se establece la comunicación serial a 115 200 baudios, se configura la resolución del ADC a 12 bits y se establece la atenuación de las entradas analógicas
- **Adquisición de las señales musculares:** En la función *loop()* se utiliza *analogRead()* para obtener continuamente los valores digitales correspondientes a los cuatro canales.
- **Filtrado Digital:** Se implementó un filtro EMA con un coef. de suavizado de 0.15 con el propósito de reducir las fluctuaciones rápidas de la señal
- **Transmisión y visualización:** Los valores filtrados se envían mediante instrucciones *Serial.print()*, identificando cada canal como EMG_1, EMG_2, EMG_3 y EMG_4 para permitir su representación gráfica

3.3 Condiciones de operación:

- El programa requiere que los sensores sEMG se encuentren conectados a las entradas analógicas correspondientes del ESP32 y que sus señales de salida estén acondicionadas para mantenerse dentro del rango de tensión permitido por el microcontrolador.
- El sistema está diseñado para operar continuamente mientras el ESP32 permanezca alimentado, realizando la adquisición, el filtrado y la transmisión de las mediciones de los cuatro canales.

3.4 Evidencia de la implementación:

- En la Fig.4 se presenta el código desarrollado en Arduino IDE, donde se observa la configuración de los canales de adquisición, el filtro digital implementado y las instrucciones utilizadas para transmitir las señales mediante comunicación serial




```ino
// Definición de pines para los 4 sensores sEMG (Todos pertenecientes al ADC1)
const int EMG_PINS[4] = {34, 35, 32, 33};

// Factor de corrección del divisor de tensión (Ejemplo: 5V a 3.3V -> ~1.515)
const float DIVISOR_FACTOR = 1.515;

// Arreglo para almacenar los valores filtrados de cada canal
float emgFiltrado[4] = {0, 0, 0, 0};

// Factor de suavizado EMA (0.05 = muy suave/lento, 0.2 = respuesta más rápida)
const float ALPHA = 0.15;

void setup() {
  Serial.begin(115200);

  // Configuración de resolución ADC a 12 bits (0 a 4095)
  analogReadResolution(12);

  // Ajuste de atenuación para el rango completo (0V a ~3.3V) en cada pin
  for (int i = 0; i < 4; i++) {
    analogSetPinAttenuation(EMG_PINS[i], ADC_11db);
    pinMode(EMG_PINS[i], INPUT);
  }
}

void loop() {
  // Lectura y filtrado de los 4 canales
  for (int i = 0; i < 4; i++) {
    int lecturaCruda = analogRead(EMG_PINS[i]);

    // Filtro de suavizado (EMA)
    emgFiltrado[i] = (ALPHA * lecturaCruda) + ((1.0 - ALPHA) * emgFiltrado[i]);
  }

  // Formato multicanal compatible con el Serial Plotter de Arduino IDE
  Serial.print("EMG_1:");
  Serial.print(emgFiltrado[0]);
  Serial.print("\t");

  Serial.print("EMG_2:");
  Serial.print(emgFiltrado[1]);
  Serial.print("\t");

  Serial.print("EMG_3:");
  Serial.print(emgFiltrado[2]);
  Serial.print("\t");

  Serial.print("EMG_4:");
  Serial.println(emgFiltrado[3]);

  delay(10); // Muestreo a ~100 Hz
}
```

Figura 4. Código de módulo de sensores sEMG

La implementación actual constituye una primera etapa del subsistema de programación y control. Como actividades pendientes se consideran la verificación de las señales musculares adquiridas, la definición de los umbrales de activación y la integración de los datos del ESP32 con el módulo de seguimiento corporal mediante visión artificial, con el propósito de desarrollar posteriormente la retroalimentación del sistema de rehabilitación.